

Organización de carpetas del proyecto

19 de septiembre de 2026

Índice

1. Idea general	1
2. Las carpetas numeradas	2
2.1. 01_webscrap_prep	2
2.2. 02_dataprep	2
2.3. 03_models	2
2.4. 04_figures	3
2.5. 05_sensibilidad_umbral	4
3. output/	4
4. shiny_boxplots/	4
5. Orden de corrida	4

1. Idea general

El proyecto tiene tres zonas dentro de **interno/**:

- **Producción**: las carpetas numeradas (01_webscrap_prep, 02_dataprep, 03_models, 04_figures, 05_sensibilidad_umbral), output/ y shiny_boxplots/, todas en la raíz de **interno/**. Es la versión que corre hoy, con el método del umbral (v2) para elegir el sku representativo de cada alimento.
- **v1/**: una copia completa y aparte del pipeline, congelada en el método anterior (el sku más parecido a 500g/1000ml). Tiene su propia versión de cada carpeta numerada, su propio output/ y su propio shiny_boxplots/.
- **docs/**: los documentos del proyecto (**reporte_proyecto** y este). Se leen de los resultados pero ninguna carpeta los lee a ellos.

v1/ no se toca ni se actualiza. Se mantiene solo como referencia para comparar contra el método nuevo; cualquier cambio al pipeline se hace únicamente en las carpetas de producción.

Dentro de cada carpeta numerada, el orden de los scripts (00_, 01_, 02_...) es el orden en que se corren: cada uno lee lo que dejó el anterior y escribe en output/. Hay una excepción, que está en la sección de orden de corrida.

2. Las carpetas numeradas

2.1. 01_webscrap_prep

Convierte los datos crudos del scraper en el panel de precios.

Script	Qué hace
00_csv_a_rds.R	Convierte los csv del scraper a .rds, uno por mes.
01_construccion_panel.R	Limpia los datos, arma el panel alimento-ciudad-sku y elige el sku representativo de cada uno.
02a_lista_alimentos.R	Decide qué alimentos quedan en cada ciudad según su cobertura de fechas.
02b_panel_balanceado.R	Deja el panel con solo los alimentos que quedaron.

La lógica que se repite o que es una regla de método vive aparte, en **aux-functions/**:

Archivo	Qué hace
seleccionar_sku_umbral.R	Elige el sku representativo de cada alimento (menor precio por gramo bajo el umbral de tamaño, con cobertura de fechas).
umbral_fechas_grupo.R	Decide qué alimentos pasan el umbral de fechas, medido contra los días de su grupo en cada ciudad.
mapeo_tcac.R	Cruza los nombres de SIPSA con el mapeo a la TCAC. Lo usan 02a (grupo de cada alimento) y 02_dataprep (composición).

Es la única diferencia real entre v1 y producción en esta carpeta: cambian las reglas de elección del sku y de cobertura, no el resto del proceso.

2.2. 02_dataprep

Le pega al panel de precios la composición nutricional de cada alimento y prepara los requerimientos del hogar representativo.

Script	Qué hace
00_tcac_composicion.R	Une la lista de alimentos con la composición nutricional de cada uno.
01_panel_mensual_tcac.R	Agrega el panel a nivel mensual, ya con la composición pegada.
02_eer_*members.R	Calcula el requerimiento energético del hogar representativo, según su número de miembros.
03_gaba_ajustado.R	Ajusta los intercambios de la guía alimentaria a la energía de cada ciudad.

Las subcarpetas **eer/**, **household eer/** y **unidades/** guardan las tablas base y los resultados intermedios de esos scripts. v1 reutiliza el mismo cálculo de requerimiento que producción; solo corre la parte que sí depende del método de precio.

2.3. 03_models

Aquí corren los modelos.

Script	Qué hace
01_coca.R	Corre el modelo de coca por ciudad-mes.
02_cona.R	Corre el modelo de cona por ciudad-mes.
03_cord.R	Corre el modelo de cord por ciudad-mes.
031_coahd.R	Corre el modelo de coahd por ciudad-mes.
04_cc_cona.R	Variante adicional; no forma parte del flujo actual.
05_models_hcost.R	Junta los costos de los cuatro modelos en el costo del hogar y el costo per cápita (día, mes y año).

La lógica de cada modelo (cómo arma y resuelve el problema) vive separada en **aux-functions/**, aparte de los scripts, que solo cargan datos, llaman al modelo y guardan el resultado:

Archivo	Qué hace
CoCA_paper.R, CoNA_paper.R	Las funciones de coca y cona.
CoRD_Herforth.R, CoAHD_Herforth.R	Las funciones de cord y coahd.
gaba_insumos.R	Grupos de la guía alimentaria, número de alimentos por grupo, lista de alimentos excluidos (ajo y ají) y armado de los insumos que comparten cord y coahd.

Cada modelo guarda su salida en su propia subcarpeta (**coca/**, **cona/**, **cord/**, **coahd/**), en **.rds** (para seguir trabajando en R) y **.xlsx** (para revisar a mano). v1 solo corre coca y cona: es lo mínimo necesario para comparar contra producción.

2.4. 04_figures

Arma las gráficas.

Script	Qué hace
00_fig_config.R	Estilo visual, colores de los modelos, rutas compartidas y la función que guarda cada figura en 4K (3840 px de ancho). Lo cargan todos los demás scripts.
02_fig_costos_modelos.R	Barras de costo por ciudad para los cuatro modelos: por miembro, per cápita y por 1000 kcal.
03_fig_cona_composicion.R	Composición de la dieta de cona.
04_fig_modelos_costos.R	Costo per cápita de los cuatro modelos juntos y mapa de primas (cuántas veces cuesta cada dieta frente al CoCA y al CoNA).
05_fig_cord_coahd_composicion.R	Composición de cord y coahd por grupo y por alimento elegido.
06_fig_cona_restricciones.R	Restricciones nutricionales activas y mapa de calor de los precios sombra de cona.
07_fig_adecuacion.R	Adecuación nutricional de la dieta de cada modelo.

01_series_alim.R es un script antiguo que ya no corre con el panel actual, porque busca una columna (**sku_code**) que el panel diario ya no trae.

En **aux-functions/**, **modelos_costos.R** lee los resultados de los cuatro modelos y calcula el costo per cápita, y **food_groups_gaba.R** tiene la clasificación de alimentos por grupo que usa el color de las figuras de cona. Esta última es compartida con v1: si cambia el conjunto de alimentos que selecciona el modelo, hay que revisarla para que ningún alimento quede sin asignar.

2.5. 05_sensibilidad_umbral

Prueba qué pasa con los costos si cambia la regla de cobertura de fechas. No forma parte del flujo principal: usa los scripts de `02_dataprep` y `03_models` tal como están y guarda sus resultados aparte, en `output/sensibilidad_umbral/`.

Script	Qué hace
<code>01_cobertura_alimentos.R</code>	Calcula los días con dato de cada alimento en cada ciudad y qué alimentos pasarían con cada regla.
<code>02_correr_regla.R</code>	Corre panel mensual y los cuatro modelos con los alimentos de una regla (se corre una vez por regla, desde la terminal).
<code>03_tablas_sensibilidad.R</code>	Junta los resultados en las tablas que usa el reporte y valida que la regla actual reproduzca los modelos.

3. output/

Carpeta	Contenido
<code>paneles/</code>	Panel de precios, crudo, filtrado, balanceado y mensual.
<code>lista_alimentos/</code>	Qué alimentos quedaron en cada ciudad y por qué.
<code>tcac/</code>	Composición nutricional de los alimentos.
<code>gabas/</code>	Intercambios y energía de la guía alimentaria, base y ajustados por ciudad.
<code>figuras/</code>	Las figuras finales, en .pdf y .png: costos, composición, restricciones y adecuación.
<code>sensibilidad_umbral/</code>	Resultados de las pruebas de la regla de fechas, una carpeta por regla.

`v1/output/` tiene la misma estructura, con los resultados calculados con el método antiguo.

4. shiny_boxplots/

App de Shiny con los boxplots de precio por gramo, alimento por alimento, con la opción de ver todo el panel o solo lo que el modelo selecciona como óptimo. Hay una copia en la raíz (producción) y otra en `v1/`, cada una publicada por separado en shinyapps.io. La carpeta `rsconnect/` dentro de cada una guarda los metadatos de esa app; hay que mantenerlos para poder redeployar a la misma URL en vez de crear una app nueva. Cuando cambia el panel o los resultados hay que correr `data_prep.R` y volver a publicar.

5. Orden de corrida

Producción se corre en este orden. El orden importa en un punto: `00_tcac_composicion.R` lee la lista de alimentos que produce `02a`, así que va después de `02a` y `02b` aunque viva en otra carpeta.

Paso	Scripts
1. Precios	<code>01_webscrap_prep</code> : <code>00_csv_a_rds</code> , <code>01_construccion_panel</code> , <code>02a_lista_alimentos</code> , <code>02b_panel_balanceado</code>
2. Composición y hogar	<code>02_dataprep</code> : <code>00_tcac_composicion</code> , <code>01_panel_mensual_tcac</code> , <code>02_eer_3members</code> , <code>03_gaba_ajustado</code>
3. Modelos	<code>03_models</code> : <code>01_coca</code> , <code>02_cona</code> , <code>03_cord</code> , <code>031_coahd</code> , <code>05_models_hcost</code>

4. Figuras	04_figures: 02, 03, 04, 05, 06 y 07
5. Sensibilidad	05_sensibilidad_umbral: 01_cobertura_alimentos, 02_correr_regla (una vez por regla) y 03_tablas_sensibilidad
6. App	shiny_boxplots/data_prep.R, y publicar si se quiere refrescar la app

Los scripts de **eer/** (requerimientos de energía por ciudad) solo hay que correrlos cuando cambian los datos de población o antropometría.

v1/ se corre igual, pero con los scripts que están dentro de **v1/**. No hay que mezclar rutas entre las dos versiones: un script de producción siempre lee y escribe dentro de **interno/**, y uno de **v1** siempre dentro de **interno/v1/**.